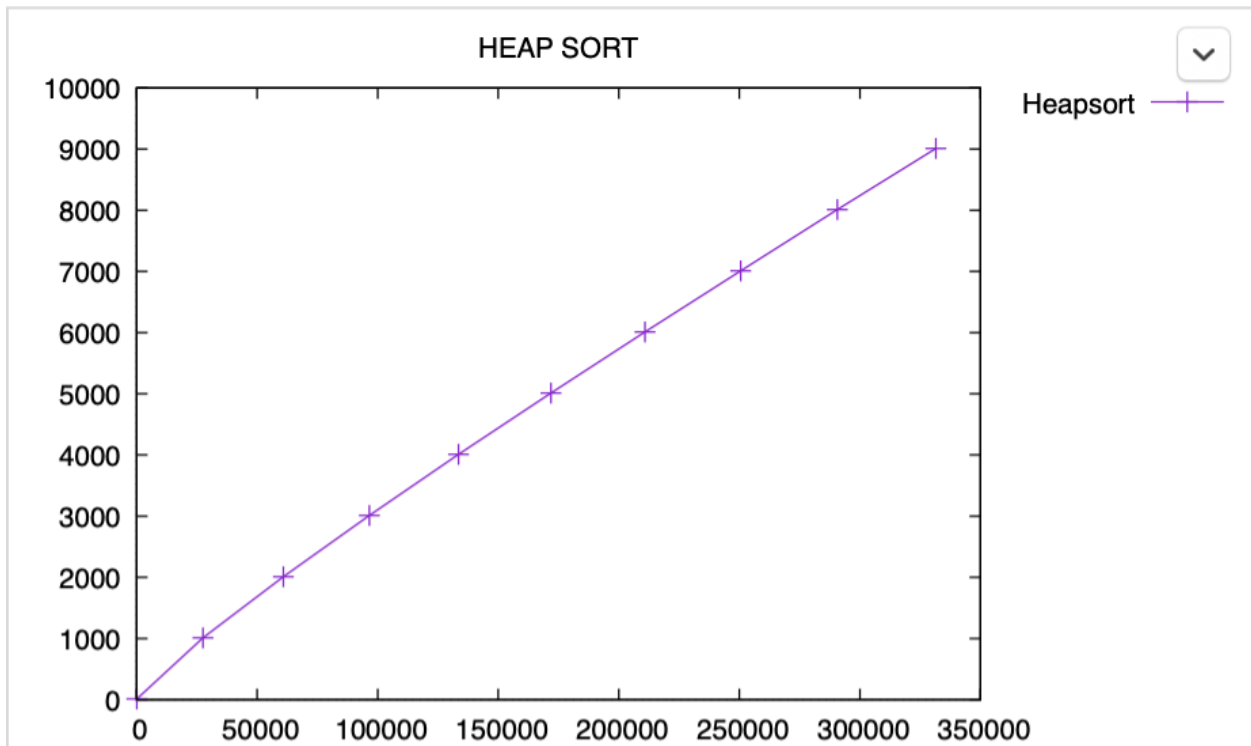


Assignment 3 Writeup - Sorting: Putting your affairs in order

Introduction:

This program implements the sorting algorithms Insertion Sort, Shell Sort, Heap Sort, and recursive Quicksort. The test harness will test each of these sorts on an array of pseudo random elements. Analysis and finding are presented in this writeup pdf.

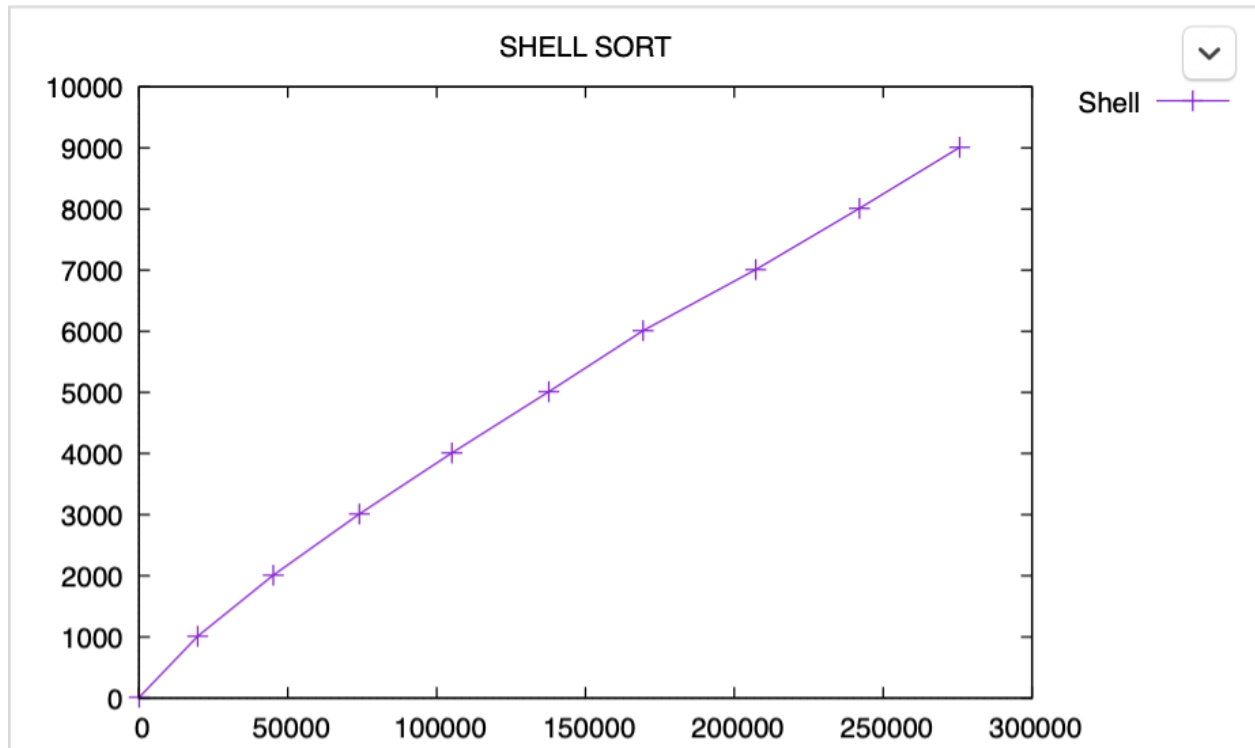
Results/Analysis:



This graph represents the moves performed by Heap sort. The x axis is the number of elements and the y axis is the moves. Heapsort is a comparison-based sorting algorithm. Although slower than quicksort, it has the advantage of a worst-case $O(n \log n)$ runtime and has the storage efficiency similar to that of quicksort. Heapsort locates the largest value and places it in the final array position. Then it locates the next largest

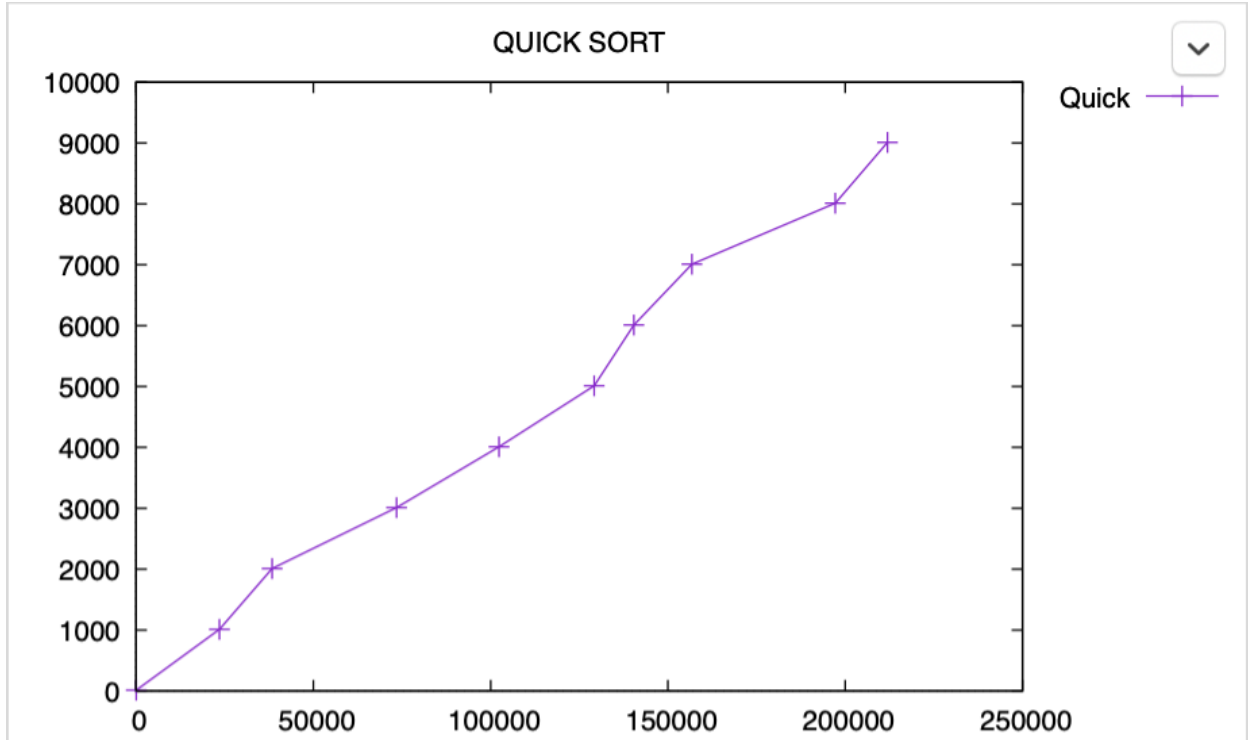
Assignment 3 Writeup - Sorting: Putting your affairs in order

value and places it in the next-to-last array position and so forth. Heapsort uses a very efficient algorithm to locate the array values to be moved.



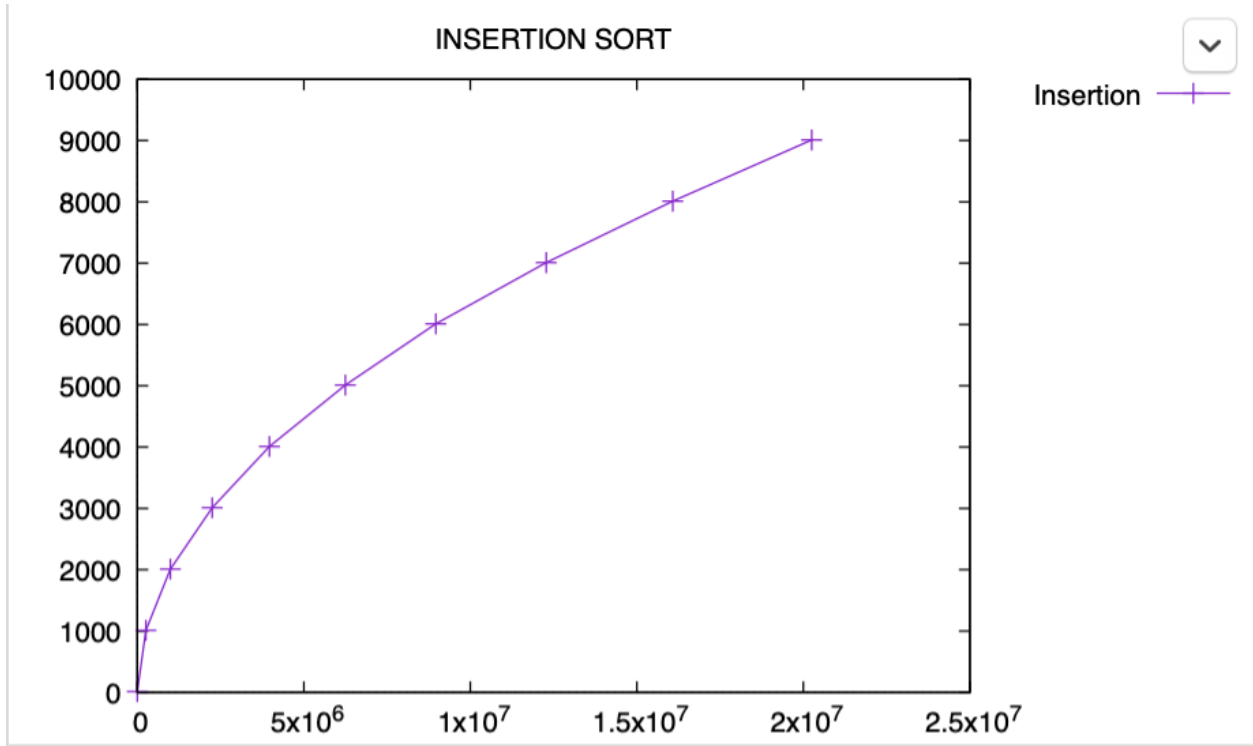
This graph represents the moves performed by Shell sort. The x axis is the number of elements and the y axis is the moves. Shell sort improves on the insertion sort by breaking the original list into a number of smaller sublists, each of which is sorted using an insertion sort. Instead of breaking the list into sublists of contiguous items, the shell sort uses a gap to create a sublist by choosing all items that are i items apart. I was surprised to find out that shell sort is more efficient than insertion sort, considering it does a complete insertion sort as the last step. It turns out, however, that this final insertion sort does not need to do very many comparisons since the list has been pre-sorted by earlier incremental insertion sorts. Basically, each pass produces a list that is more sorted than the previous one. This makes the shell sort very efficient.

Assignment 3 Writeup - Sorting: Putting your affairs in order



This graph represents the moves performed by Quick sort. The x axis is the number of elements and the y axis is the moves. Quicksort is a divide-and-conquer algorithm. It partitions arrays into two sub-arrays by selecting an element from the array and designating it as a pivot. Elements in the array that are less than the pivot go to the left sub-array, and elements in the array that are greater than or equal to the pivot go to the right sub-array. As you can see from the graph, Quicksort was the fastest sorting algorithm that I implemented.

Assignment 3 Writeup - Sorting: Putting your affairs in order



This graph represents the moves performed by Insertion sort. The x axis is the number of elements and the y axis is the moves. Insertion sort is a comparison sort in which the sorted array is built one entry at a time. It is much less efficient on large lists than more advanced algorithms such as quicksort, heapsort, etc. However, insertion sort provides several advantages: it has a very simple implementation, it is efficient for small data sets, it requires a constant amount $O(1)$ of additional memory space.